Fashion-MNIST Classification with a Three-Layer Neural Network Implemented from Scratch

Computer Vision HW1

Abstract

This report presents a from-scratch three-layer multilayer perceptron for Fashion-MNIST classification. The full training pipeline is implemented with NumPy, including forward propagation, backward propagation, loss computation, optimization, checkpoint selection, evaluation, and visualization. The experiments are organized in stages to examine the effects of learning rate, activation function, weight decay, hidden dimension, and scheduler choice. Short-run experiments identify a strong operating region, and a 200-epoch refinement stage is then used to select the final model. The selected configuration uses hidden dimension 512, Leaky ReLU, initial learning rate 0.3, weight decay 10^{-4}, and step decay with a 50-epoch interval. This model reaches 0.9132 validation accuracy and 0.9006 test accuracy.

1 Introduction

The purpose of this project is to train a Fashion-MNIST classifier under the constraint that all gradients and parameter updates must be implemented manually. The model is intentionally simple: a three-layer MLP without convolutional structure or framework-based autodiff. This keeps the analysis focused on optimization and representation choices rather than on architectural complexity. The central question is not whether a stronger model could do better, but how far a carefully tuned from-scratch MLP can be pushed under the assignment boundary.

Fashion-MNIST is suitable for this study because it remains visually ambiguous despite its small scale. Several classes are easy to distinguish by silhouette, while others overlap strongly in global shape and low-resolution texture. This makes the dataset large enough for meaningful optimization analysis and small enough for repeated controlled experiments.

2 Method

The classifier maps a 784-dimensional flattened input to ten output logits through two hidden layers and one output layer. Hidden width is configurable, and the nonlinearity can be switched across multiple activation functions. Training uses numerically stabilized softmax cross-entropy and SGD with optional L2 regularization. Learning-rate scheduling is implemented independently from the optimizer.

The pipeline does not use patience-based early stopping. Instead, training runs for a fixed number of epochs and retains the checkpoint with the highest validation accuracy. This policy matches the homework requirement of validation-based best-checkpoint saving and proved important in long-run experiments, where the best model did not coincide with the final epoch.

Images are normalized to the range $[0, 1]$, and the original training split is divided into training and validation subsets with a validation ratio of 0.1. Validation data is used for model selection and hyperparameter comparison, while the test set is reserved for final evaluation only.

3 Experimental Protocol

The experiments were carried out in two stages. The first stage used 20-epoch runs to identify a strong operating region quickly. The second stage used longer training to check whether the short-run ranking remained stable after more complete optimization. This staged design reduced the number of expensive runs while still allowing the final model to be selected on the basis of longer-horizon evidence rather than on short-run convenience.

Table 1 summarizes the final selected configuration.

Table 1: Final model configuration.

Parameter	Value
Input dimension	784
Hidden dimension	512
Output dimension	10
Activation	Leaky ReLU
Batch size	128
Initial learning rate	0.3
Weight decay	10^{-4}
Scheduler	Step decay
Step size	50
Gamma	0.5
Epochs	200
Best validation epoch	81

4 Results

4.1 Learning-Rate Study

The initial question was whether the baseline learning rate was already large enough. Under the short-run ReLU setting, validation accuracy improved steadily as the rate increased from 0.01 to 0.3. This immediately showed that the original baseline range was too conservative. A second sweep tested whether values above 0.3 would continue to improve the result. The answer was negative.

A learning rate of 0.5 remained trainable but weaker than 0.3, while 1.0 collapsed to near-random performance. The useful region was therefore centered around 0.3 rather than around 0.05 or 0.1.

Table 2: Learning-rate comparison under the short-run ReLU setting.

Learning rate	Best val. acc.	Best epoch
0.01	0.8478	16
0.05	0.8813	20
0.10	0.8928	20
0.15	0.8962	20
0.20	0.8988	19
0.30	0.9010	19
0.50	0.8950	19
1.00	0.1040	3

The curves in Fig. 1 support the numerical result. Moderate increases in learning rate improved optimization speed and final validation accuracy, but the largest tested value destroyed stability. The key observation is that the useful learning-rate region extended well beyond the initial baseline. This matters because a narrower search would have produced a misleading conclusion: the baseline would have looked acceptable even though the optimizer was still operating below its effective regime. The failure at 1.0 is equally informative. It shows that the benefit of larger steps comes from approaching a narrow interval of efficient descent, not from increasing the step size without limit.

There is also a methodological point behind this comparison. Learning rate was not a secondary cosmetic adjustment in this project. It defined whether later experiments were conducted in a strong optimization region or in a weak one. Once this is recognized, the role of the learning-rate study becomes broader than simply selecting 0.3. It establishes the stability boundary of the training pipeline and makes later comparisons more trustworthy.

4.2 Activation Study

Activation ranking depended on training horizon. In the 20-epoch comparison, ReLU ranked first, but Leaky ReLU was already close. Sigmoid was clearly the weakest activation, while Tanh, ELU, Swish, and Softplus formed a middle group.

Table 3: Activation comparison at 20 epochs.

Activation	Best val. acc.	Best epoch
ReLU	0.9010	19
Leaky ReLU	0.8998	17
Tanh	0.8958	17
Swish	0.8927	18
ELU	0.8917	20
Softplus	0.8800	16
Sigmoid	0.8565	20

Under the short horizon, ReLU appeared to be the natural final choice. That ranking changed after longer optimization. A dedicated 200-epoch comparison between ReLU and Leaky ReLU showed that Leaky ReLU eventually overtook ReLU.

Table 4: Long-run activation comparison at 200 epochs.

Activation	Epochs	Best val. acc.	Best epoch
ReLU	200	0.9122	140
Leaky ReLU	200	0.9132	81

This result is small in magnitude but important in interpretation. The short-run ranking was not final. Leaky ReLU reached a slightly stronger validation peak and reached it much earlier. That difference would have been easy to dismiss if the analysis had stopped at 20 epochs, yet the longer run shows that the early ranking was incomplete rather than definitive.

The comparison is analytically useful because it separates two different notions of improvement. In short runs, the stronger activation is the one that delivers faster early progress. In long runs, the stronger activation is the one that preserves gradient quality and continues improving after the early transient. ReLU satisfied the first criterion slightly better, but Leaky ReLU satisfied the second. This is why the final choice depends on training horizon rather than on the short-run table alone.

4.3 Weight Decay

The weight-decay study fixed ReLU, hidden dimension 128, and learning rate 0.3. The gain from regularization was modest, but the trend was consistent: mild regularization was better than both stronger regularization and none at all.

Table 5: Weight-decay comparison.

Weight decay	Best val. acc.	Best epoch
0	0.9012	19
10^{-4}	0.9023	20
5×10^{-4}	0.9010	19
10^{-3}	0.8978	20

The practical interpretation is straightforward, but the result is still worth unpacking. Weight decay was not the dominant source of improvement in this project; the main performance gains came from learning-rate and capacity choices. Even so, regularization still mattered as a refinement step. The fact that zero decay and 5×10^{-4} remain close to the optimum shows that the system is not hypersensitive to this parameter once it is already in a good operating region. The fact that 10^{-3} degrades performance shows where the regularization boundary begins to suppress useful fit.

This makes the selected value of 10^{-4} meaningful in a specific sense. It should not be interpreted as a universally superior penalty, but as the weakest tested regularizer that consistently improved generalization under

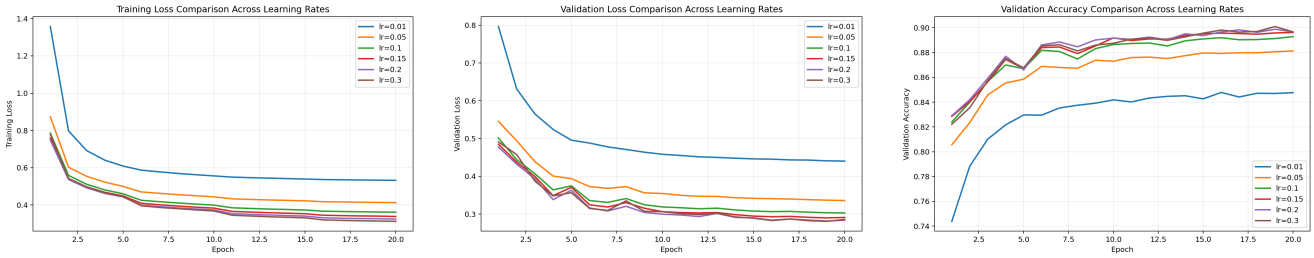

Figure 1: Learning-rate comparison. Left: training loss. Middle: validation loss. Right: validation accuracy.

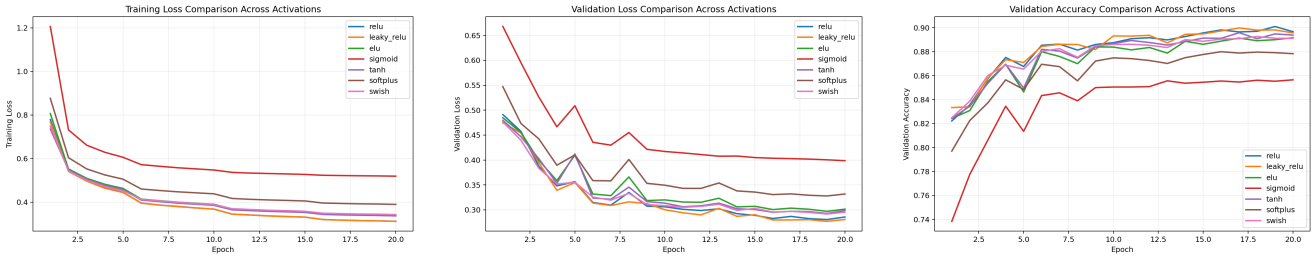

Figure 2: Short-run activation comparison. Left: training loss. Middle: validation loss. Right: validation accuracy.

the chosen optimizer and model width. That is exactly the scale of decision that matters in the final refinement stage.

4.4 Hidden Dimension

Increasing hidden dimension improved validation performance up to a point. The trend from 64 to 512 was consistently upward, which justified moving beyond the original 128-dimensional baseline. However, the extended comparison showed that the trend did not continue indefinitely.

Table 6: Hidden-dimension comparison.

Hidden dim.	Best val. acc.	Best epoch	Time (s)
64	0.8967	17	16.38
128	0.9023	20	27.05
256	0.9043	19	68.65
512	0.9100	20	154.74
768	0.9078	19	307.92
1024	0.9077	20	372.23

The interesting point is not only that 512 is best, but also why the larger models were not adopted. The accuracy gain saturates beyond 512, while training cost continues to rise sharply. This creates a clean capacity argument: wider models initially help, but additional width eventually brings diminishing returns without compensating improvements in validation quality.

This result matters because it distinguishes "largest" from "best." If the study had stopped at 512, one could still argue that even larger widths might continue improving accuracy. The additional 768 and 1024 runs remove that ambiguity. They show that once the model is already wide enough to separate most classes effectively, extra capacity is no longer translated into better validation behavior. Instead, it is translated primarily into extra computation.

The time column therefore carries analytical weight rather than serving as a side note. The move from 256 to 512 remains defensible because the validation gain is still meaningful. The move from 512 to 768 or 1024 is much harder to justify, because the cost increase is dramatic while the validation result does not improve. This is why hidden dimension 512 is the strongest choice in both statistical and practical terms.

4.5 Scheduler

Scheduler comparison was carried out under the strongest short-run setting with hidden dimension 512. Step decay produced the best validation result, with cosine decay only slightly behind. No decay was weaker, and exponential decay fell much further behind.

Table 7: Scheduler comparison.

Scheduler	Best val. acc.	Best epoch
None	0.9033	19
Step	0.9100	20
Exponential	0.8852	9
Cosine	0.9093	18

Step and cosine are both viable choices, but step decay retains a small edge under the short-run comparison. The final long-run setup therefore keeps step decay while extending its interval to 50 epochs so that the schedule remains meaningful over 200 epochs.

The scheduler comparison clarifies what the schedule is actually contributing. It is not rescuing a weak configuration. The model is already trainable without decay, as the no-decay result shows. The scheduler instead shapes the later optimization phases and determines how efficiently the run converts a strong initial regime into a better validation checkpoint. This is why the difference

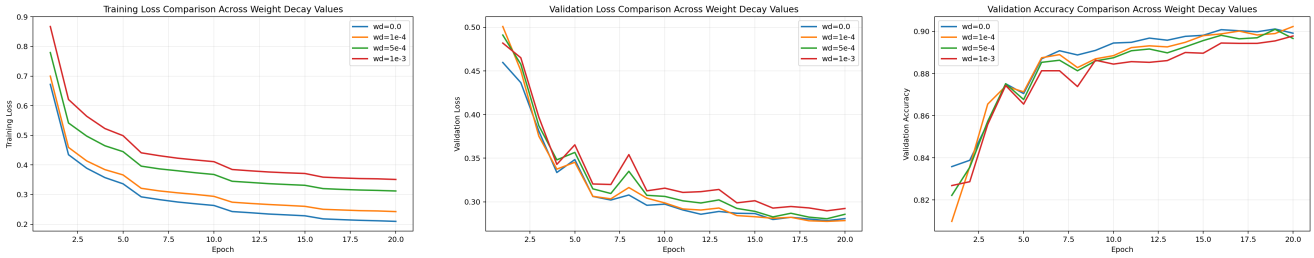

Figure 3: Weight-decay comparison. Left: training loss. Middle: validation loss. Right: validation accuracy.

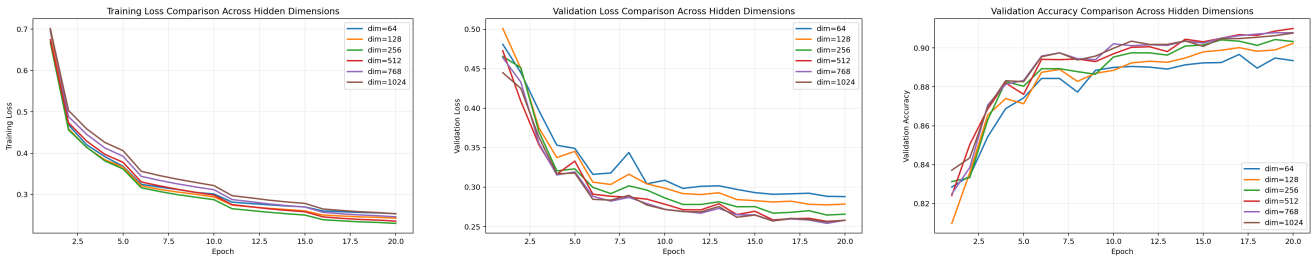

Figure 4: Hidden-dimension comparison. Left: training loss. Middle: validation loss. Right: validation accuracy.

between no decay and step decay is meaningful even though both are workable.

The weak result for exponential decay is also informative. In the tested setting, it reduces the effective learning rate too aggressively relative to the 20-epoch budget. Step and cosine avoid that problem by keeping the optimizer in a productive regime for longer. The final choice of step decay therefore reflects not only the highest score but also a schedule shape that matches the observed optimization dynamics.

4.6 Final Training Behavior

The final model uses Leaky ReLU, hidden dimension 512, learning rate 0.3, weight decay 10^{-4}, and a 200-epoch step schedule with decay every 50 epochs. The best validation result appears at epoch 81.

The 200-epoch curves show a mild form of overfitting when loss and accuracy are read together. Training loss decreases throughout the run, but validation loss reaches its minimum much earlier and then fluctuates at a higher level. Validation accuracy behaves differently: it remains high and reaches its best value at epoch 81 before entering a relatively stable plateau. This means that prolonged optimization continues to improve fit to the training set without continuing to improve validation calibration in the same way. The effect is therefore better described as mild long-run overfitting than as catastrophic degradation.

This distinction is important because it changes how the final run should be interpreted. If only validation accuracy were considered, the run would appear broadly stable and successful through much of the later training period. If only validation loss were considered, one might argue that the useful part of training ended much earlier. Neither view is sufficient alone. The combination of the two metrics gives a more careful reading: the classification boundary remains strong, but the probability calibration becomes less favorable after the early low-loss region.

There is also a broader point here. The long run was not only used to squeeze out a slightly higher score. It exposed the final activation ranking and revealed that the best validation-accuracy checkpoint and the best validation-loss checkpoint are not the same object. That is a stronger justification for checkpoint selection than a simple statement that "the best epoch was not the last one."

4.7 Final Test Performance

The final selected checkpoint achieved 0.9006 accuracy on the held-out test set.

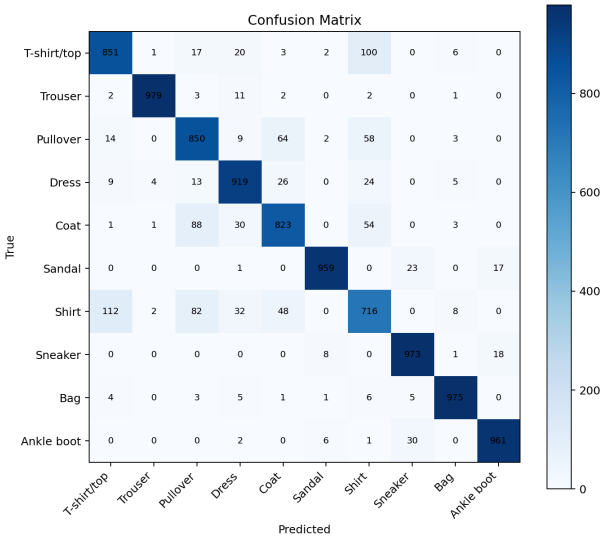

Figure 7: Confusion matrix on the final test set.

Test performance is uneven across classes. Trouser, sneaker, bag, and ankle boot are recognized with very high reliability, while shirt remains the weakest class. The confusion matrix shows that the dominant failure

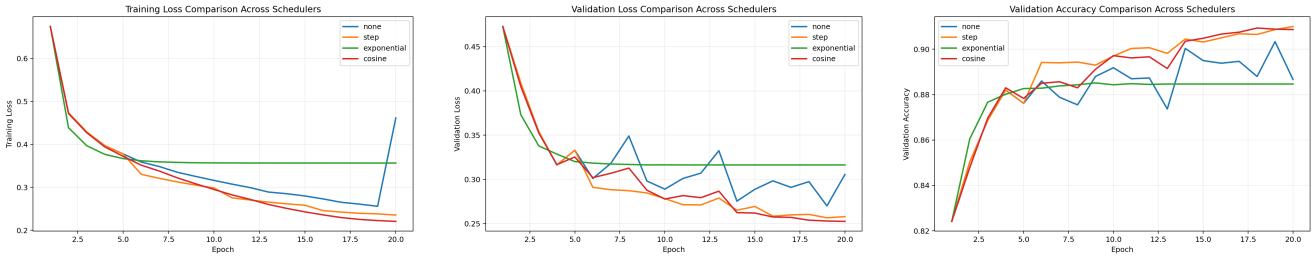

Figure 5: Scheduler comparison. Left: training loss. Middle: validation loss. Right: validation accuracy.

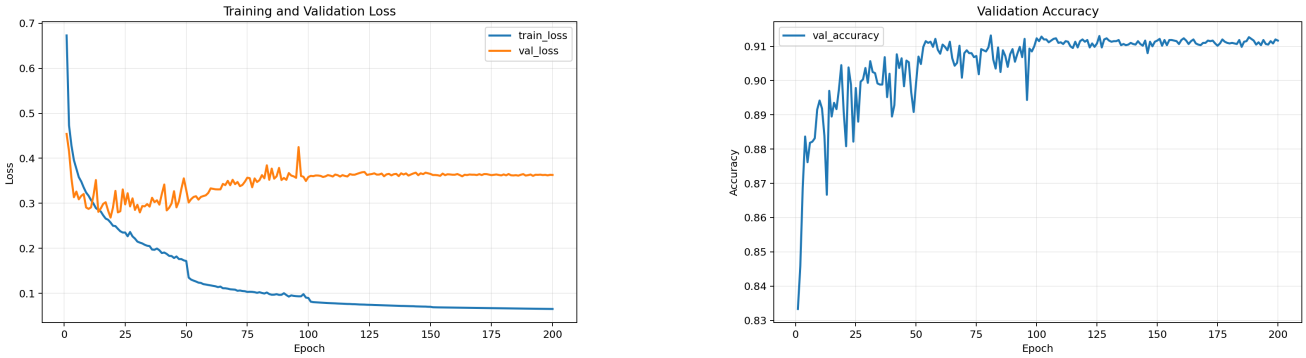

Figure 6: Final training curves. Left: training and validation loss. Right: validation accuracy.

pattern is not random noise but overlap among visually similar upper-body categories. This is an important limitation of the final model and should be treated as part of the result rather than as an exception to it.

The test result is useful for two reasons. First, it confirms that the staged validation-based search transferred to held-out data rather than simply over-optimizing the validation split. Second, it shows where the remaining error budget is concentrated. The model does not fail uniformly; it fails on specific class groups that remain hard to separate after flattening. That makes the final test section more informative than a single scalar accuracy by itself.

4.8 First-Layer Weights

The first-layer weights, reshaped into 28 by 28 maps, reveal structured spatial templates rather than diffuse noise.

Many neurons emphasize central vertical regions, lower-body contours, or broad edge transitions around clothing boundaries. Even without convolutions, the network still learns coarse spatial regularities. The first layer therefore acts less like a generic dense transform and more like a bank of large-scale garment templates.

This figure helps because it makes the model less abstract. The final classifier is not merely exploiting an opaque latent space. The first hidden layer has acquired visible structure, even though that structure is coarser than what would be expected in a convolutional model. This does not imply that each neuron corresponds to a clean semantic part, but it does show that the model has learned reusable spatial templates rather than only unstructured global weights.

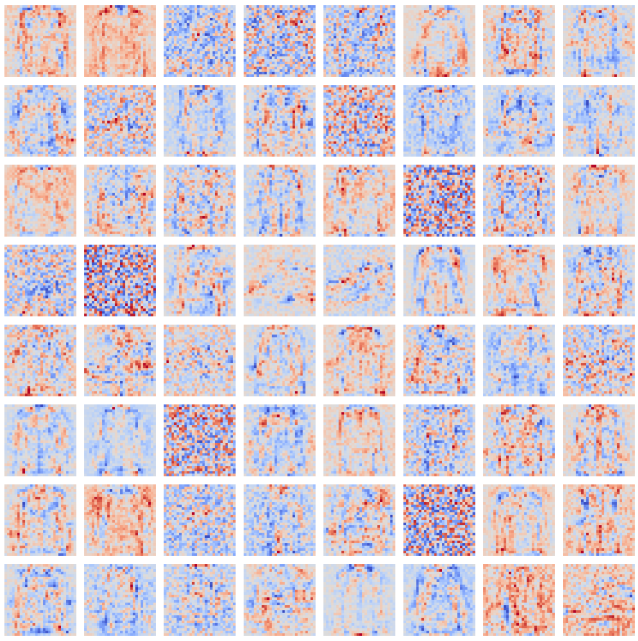

Figure 8: Visualization of first-layer weights.

4.9 Misclassified Examples

The failure cases support the pattern already visible in the confusion matrix. Shirt is frequently confused with T-shirt/top, pullover, or coat, and some dress-like and coat-like instances remain ambiguous at 28 by 28 resolution. The errors are therefore semantically plausible, not arbitrary. This matters because it indicates that the model learns meaningful boundaries but remains limited by the loss of local spatial structure after flattening.

The useful point here is not simply that the classifier makes mistakes. A weak model often produces errors that are difficult to interpret. In contrast, many of the

mistakes shown here are visually understandable. That strengthens the claim that the representation is already meaningful and that the remaining weakness is largely architectural rather than procedural.

Figure 9: Representative misclassified test samples.

4.10 Hidden-Feature Structure

The t-SNE projection provides a compact visual summary of the learned feature space. Several classes form tight clusters, particularly those with strong silhouette signatures. Upper-body garment classes remain more entangled. This agrees with both the confusion matrix and the error analysis, which suggests that the final representation is discriminative but not uniformly separated across all categories.

This figure is useful not because t-SNE is itself an evaluation metric, but because it visually links the earlier observations. The classes that were easiest in the confusion matrix tend to occupy more isolated regions, while the classes that generated the most plausible mistakes remain mixed. In that sense, the t-SNE projection acts as a bridge between the scalar results and the qualitative interpretation.

Figure 10: t-SNE projection of penultimate hidden features.

5 Conclusion

The final model was obtained through staged search rather than through a single uncontrolled sweep. The experiments lead to several consistent conclusions. First, the effective learning-rate region extended well beyond the initial baseline, and the useful maximum in the tested range was 0.3. Second, mild weight decay outperformed both stronger regularization and none at all. Third, hidden-dimension gains saturated beyond 512, which made 512 the strongest width among the tested models. Fourth, step decay remained slightly stronger than cosine under the short-run comparison and provided a suitable backbone for long training. Finally, the activation ranking changed with training horizon: ReLU led in short runs, but Leaky ReLU slightly surpassed it after 200 epochs.

The final checkpoint achieved 0.9132 validation accuracy and 0.9006 test accuracy. The remaining weakness lies in the separation of visually similar upper-body categories after flattening. Even so, the implementation satisfies the assignment requirements and produces a complete, reproducible experimental pipeline built entirely without automatic differentiation.

External Resources. GitHub repository: https://github.com/yuheng-li-ai/CV_HW1. Model-weight download link: To be filled.